Inside the Neural Network — How "Paint Peeling" Becomes [0.241, -0.812, ...]

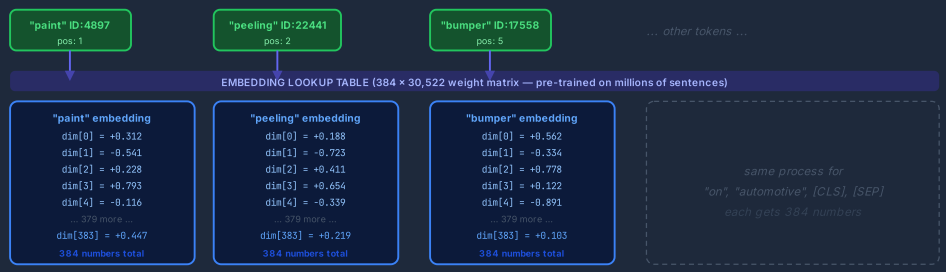
Follow the actual data values step by step through every layer of the all-MiniLM-L6-v2 model

Token IDs at every node | Weight values on connections | Activation function outputs | 384-D final vector

Stage 1 — Tokenization: Words Become Token IDs

Stage 1: The tokenizer splits the sentence into tokens. Each token gets a unique integer ID. Position numbers tell the model where each word appears in the sentence.

Stage 2 — Token Embedding: Each ID Becomes a 384-Number Vector



Stage 2: Each token ID is looked up in a massive pre-trained table. The word "paint" maps to a unique 384-number embedding, "peeling" maps to another, "bumper" to another — all learned from training on millions of sentences.

Stage 3 — Self-Attention: Tokens Understand Context by Talking to Each Other

This is the most important stage. Each word does not just look at itself — it "attends" to every other word in the sentence and adjusts its own numbers based on how related they are. This is why "paint" near "bumper" means something very specific (a surface coating defect), not paint as in artwork.

Stage 3: The Attention Layer — "peeling" scores 0.95 attention weight on "bumper" and 0.69 on "paint". This means "peeling" absorbs the context of "bumper" into its own numbers. Then Mean Pooling averages all tokens into the final 384-number sentence vector.

RAG — Retrieval Augmented Generation: Step by Step

How Quality AI retrieves the right past solution when you ask "How was paint peeling solved before?"

The Complete RAG Pipeline in One View

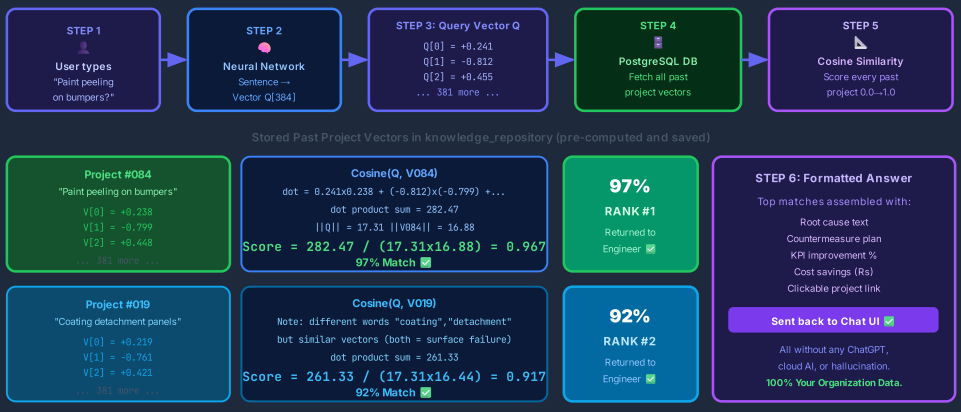


Figure: Complete RAG pipeline — from user's question to verified past solution, showing actual vector numbers, cosine computation, and scored results for two matching projects.

6-Step RAG Process at a Glance

1

Engineer Types the Question

Text sent to `POST /api/rag/chat` with JWT token.
Query: `"How was paint peeling solved before?"`

2

Auth + Tenant Lock

JWT verified. Every DB query locked to `org_id=5`. Company A can never see Company B's solutions.

3

Sentence → 384-Number Vector

Neural network tokenizes, embeds, runs self-attention (tokens "talk" to each other), then mean-pools into `Q = [0.241, -0.812, 0.455, ...]`

4

Fetch All Stored Vectors from DB

Every past project in `knowledge_repository` already has its own pre-computed 384-number vector saved during data ingestion.

5

Cosine Similarity Scored for Every Project

For each stored vector V: `Score = dot(Q,V) / (||Q|| × ||V||)`. Result: Project #084 = 97%, Project #019 = 92%, Project #112 = 4%. Sorted descending.

6

Top Matches Formatted + Returned

Root cause, countermeasure, ₹ savings, and % KPI improvement from top 3-4 projects assembled into a markdown answer with clickable storyboard links.